PropTrack

Real Estate Listing & Transaction Management System

Database Design Document

Surya Tej Boddu | sub151@pitt.edu

INFSCI 2710 — Spring 2026

April 10, 2026

1. Project Overview

Domain: Residential real estate brokerage operations.

PropTrack is a database-driven information system that models the full lifecycle of a real estate transaction — from listing a property, through scheduling showings and submitting offers, to executing a contract and conducting an inspection.

The system tracks the key entities in a brokerage:

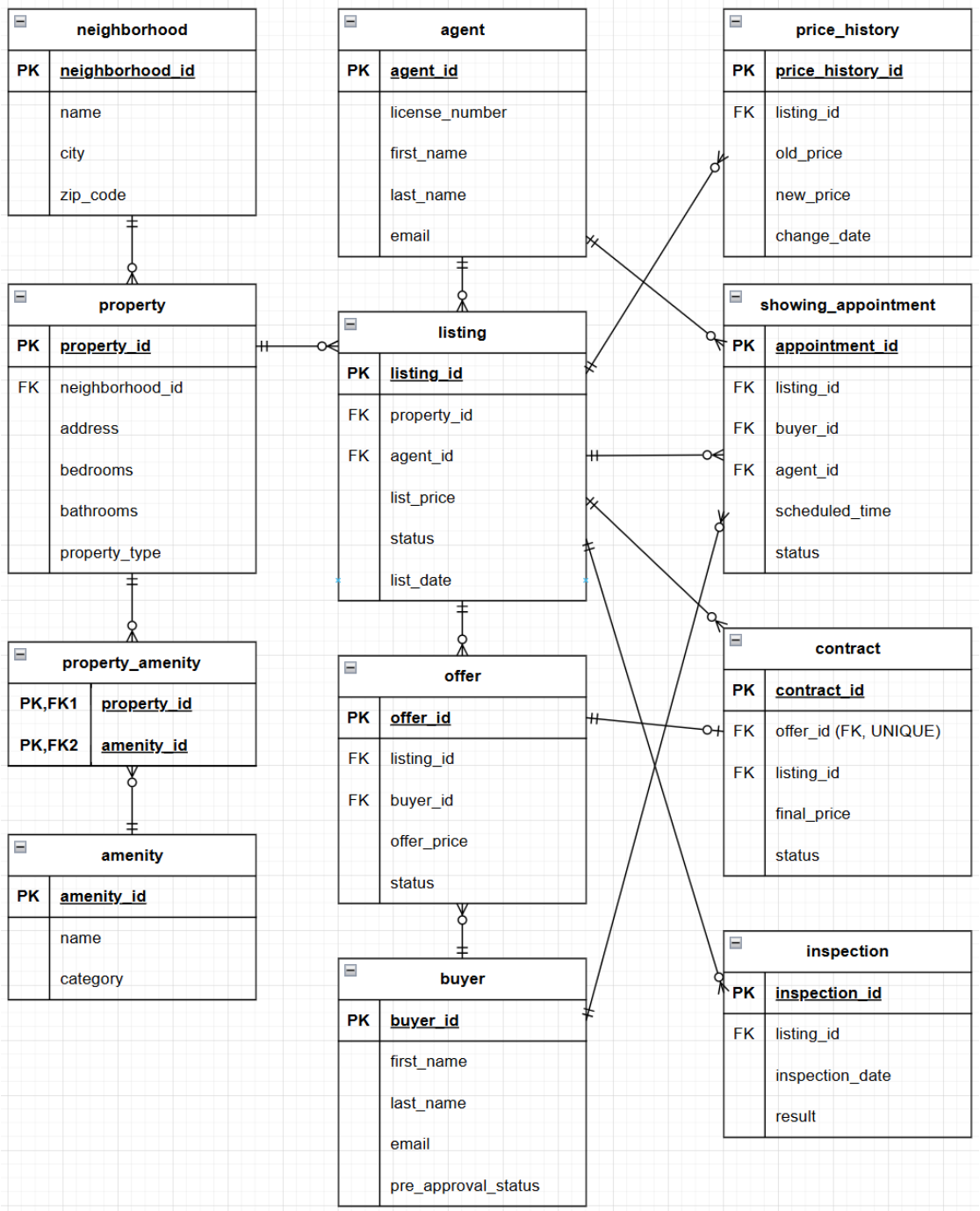
- Neighborhoods, Properties, Agents, Buyers
- Listings, Price History, Amenities
- Showing Appointments, Offers, Contracts, Inspections

Why this system is useful:

Real estate transactions involve many interdependent parties and time-sensitive state changes (e.g., a listing moving from active to under contract the moment an offer is accepted). A relational database enforces referential integrity and transactional atomicity, preventing data inconsistencies that would arise from managing these workflows in spreadsheets or disconnected tools. The accompanying web application exposes this data through a clean interface that supports the entire CRUD lifecycle.

2. Entity-Relationship (ER) Diagram

The following diagram shows all entities, their key attributes, and relationships with cardinality notation.



Entities and Relationships:

Relationship	Cardinality	Description
neighborhood → property	1 : many	A neighborhood contains many properties
property → listing	1 : many	A property can be listed multiple times
agent → listing	1 : many	An agent manages many listings
listing → price_history	1 : many	A listing can have many price changes
listing → showing_appointment	1 : many	A listing can have many showings
buyer → showing_appointment	1 : many	A buyer can attend many showings
agent → showing_appointment	1 : many	An agent hosts many showings
listing → offer	1 : many	A listing can receive many offers
buyer → offer	1 : many	A buyer can make many offers
offer → contract	1 : 1	Each accepted offer creates exactly one contract
listing → contract	1 : many	A listing can have contracts over time
listing → inspection	1 : many	A listing can have many inspections
property ↔ amenity	many : many	Via property_amenity junction table

3. Schema Description

neighborhood

Stores geographic and demographic data about each neighborhood. Separating neighborhood-level attributes avoids duplicating walkability scores and school ratings across every property row.

Column	Type	Constraints
neighborhood_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
city	VARCHAR(100)	NOT NULL
state	CHAR(2)	NOT NULL
zip_code	VARCHAR(10)	NOT NULL
walkability_score	DECIMAL(4,1)	nullable
school_rating	DECIMAL(3,1)	nullable
flood_zone	VARCHAR(10)	nullable
median_income	DECIMAL(12,2)	nullable

agent

Represents licensed real estate agents. license_number and email carry UNIQUE constraints and indexes for fast lookup.

Column	Type	Constraints
agent_id	INT	PK, AUTO_INCREMENT
license_number	VARCHAR(50)	NOT NULL, UNIQUE
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
phone	VARCHAR(20)	nullable
brokerage	VARCHAR(100)	NOT NULL
hire_date	DATE	nullable

buyer

Represents prospective buyers. Pre-approval status and budget range are attributes of the buyer, not of any specific offer.

Column	Type	Constraints
buyer_id	INT	PK, AUTO_INCREMENT

first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
phone	VARCHAR(20)	nullable
pre_approval_status	ENUM('none','pending','approved')	DEFAULT 'none'
budget_min	DECIMAL(12,2)	nullable
budget_max	DECIMAL(12,2)	nullable
preferences	TEXT	nullable

property

Represents the physical real estate unit. Separated from listing so a property can be relisted without duplicating physical attributes.

Column	Type	Constraints
property_id	INT	PK, AUTO_INCREMENT
address	VARCHAR(200)	NOT NULL
city	VARCHAR(100)	NOT NULL
state	CHAR(2)	NOT NULL
zip_code	VARCHAR(10)	NOT NULL
square_feet	INT	nullable
bedrooms	TINYINT	nullable
bathrooms	DECIMAL(3,1)	nullable
year_built	YEAR	nullable
property_type	ENUM('single_family','condo','townhouse','multi_family')	NOT NULL
neighborhood_id	INT	FK → neighborhood

listing

Represents a specific attempt to sell a property. Status is the most-queried column and carries a dedicated index.

Column	Type	Constraints
listing_id	INT	PK, AUTO_INCREMENT
property_id	INT	NOT NULL, FK → property
agent_id	INT	NOT NULL, FK → agent
list_price	DECIMAL(12,2)	NOT NULL
status	ENUM('active','under_contract','sold','withdrawn')	DEFAULT 'active'

list_date	DATE	NOT NULL
expiration_date	DATE	nullable
description	TEXT	nullable

price_history

Records every price change on a listing. Preserves full pricing history rather than overwriting list_price.

Column	Type	Constraints
price_history_id	INT	PK, AUTO_INCREMENT
listing_id	INT	NOT NULL, FK → listing
old_price	DECIMAL(12,2)	NOT NULL
new_price	DECIMAL(12,2)	NOT NULL
change_date	DATE	NOT NULL
change_reason	VARCHAR(200)	nullable

amenity

Controlled vocabulary of amenities (pool, garage, etc.). Centralizing names prevents inconsistencies and supports category filtering.

Column	Type	Constraints
amenity_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL, UNIQUE
category	VARCHAR(50)	nullable

property_amenity (junction table)

Associates properties with amenities in a many-to-many relationship.

Column	Type	Constraints
property_id	INT	PK (composite), FK → property
amenity_id	INT	PK (composite), FK → amenity

showing_appointment

Records scheduled or completed showings. scheduled_time is indexed for date-range calendar queries.

Column	Type	Constraints

appointment_id	INT	PK, AUTO_INCREMENT
listing_id	INT	NOT NULL, FK → listing
buyer_id	INT	NOT NULL, FK → buyer
agent_id	INT	NOT NULL, FK → agent
scheduled_time	DATETIME	NOT NULL
status	ENUM('scheduled','completed','cancelled')	DEFAULT 'scheduled'
feedback_notes	TEXT	nullable

offer

Records a buyer's offer on a listing. listing_id and status are indexed for the listing detail page and contract workflow.

Column	Type	Constraints
offer_id	INT	PK, AUTO_INCREMENT
listing_id	INT	NOT NULL, FK → listing
buyer_id	INT	NOT NULL, FK → buyer
offer_price	DECIMAL(12,2)	NOT NULL
contingencies	TEXT	nullable
expiration_date	DATE	nullable
status	ENUM('pending','accepted','rejected','expired','withdrawn')	DEFAULT 'pending'
submitted_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

contract

Created when an offer is accepted. offer_id is UNIQUE (1:1 with offer). listing_id stored directly to avoid a join through offer for common queries.

Column	Type	Constraints
contract_id	INT	PK, AUTO_INCREMENT
offer_id	INT	NOT NULL, UNIQUE, FK → offer
listing_id	INT	NOT NULL, FK → listing
closing_date	DATE	nullable

final_price	DECIMAL(12,2)	NOT NULL
earnest_money	DECIMAL(12,2)	nullable
contract_date	DATE	NOT NULL
status	ENUM('active','closed','cancelled')	DEFAULT 'active'

inspection

Records the result of a property inspection for a listing.

Column	Type	Constraints
inspection_id	INT	PK, AUTO_INCREMENT
listing_id	INT	NOT NULL, FK → listing
inspector_name	VARCHAR(100)	NOT NULL
inspection_date	DATE	NOT NULL
result	ENUM('pass','fail','conditional')	NOT NULL
issues_found	TEXT	nullable
report_url	VARCHAR(255)	nullable

4. Normalization Justification

First Normal Form (1NF)

All columns hold atomic, indivisible values. The preferences (buyer) and contingencies (offer) columns use TEXT for free-form notes — these are intentionally variable and cannot be meaningfully decomposed into columns without imposing an arbitrary structure.

Second Normal Form (2NF)

Every non-key column depends on the entire primary key. The only composite-key table is property_amenity, which holds no non-key columns whatsoever, so 2NF is trivially satisfied for that table. All other tables use a single-column surrogate PK (AUTO_INCREMENT INT), so partial dependency cannot occur.

Third Normal Form (3NF)

There are no transitive dependencies in any table:

- Neighborhood attributes (walkability_score, school_rating, flood_zone) live in the neighborhood table. A property references its neighborhood via FK — these values do not depend on property_id, they depend on neighborhood_id.
- Agent contact details (email, phone, brokerage) live in agent. The listing table stores only agent_id. Updating an agent's brokerage requires changing one row in agent, not every listing row they own.
- Buyer budget and approval status live in buyer. The offer table stores only buyer_id and the offer-specific price — no buyer attributes are repeated in offer.
- Contract stores listing_id directly alongside offer_id to support the application's most common contract queries and simplify joins in the contracts workflow. Because both columns are protected by foreign key constraints and offer_id remains UNIQUE, this is a documented implementation choice for efficiency rather than an uncontrolled source of inconsistency.

Conclusion: The schema satisfies the assignment's normalization expectations for a well-structured relational design, while documenting one efficiency-oriented implementation choice in the contract table. In practice, the design still avoids the common insertion, update, and deletion anomalies the project is expected to prevent.

5. Indexes

Nine indexes are created on columns that are searched, filtered, or joined most frequently in application queries.

Index Name	Table / Column(s)	Justification
idx_listing_status	listing / status	Every property search and listing page filters by status (active, sold, etc.). Most frequent WHERE clause in the entire app.
idx_listing_agent	listing / agent_id	The agent dashboard and agent-performance report join listing → agent on this column.
idx_listing_date	listing / list_date	Date-range queries (recent listings, market trend reports) filter and sort on list_date.
idx_property_nbhd	property / neighborhood_id	Every listing detail page joins property → neighborhood using this FK column.
idx_agent_email	agent / email	Agent lookup by email is the primary search key; the UNIQUE constraint already implies an index but an explicit one is created for clarity.
idx_buyer_email	buyer / email	Mirrors the agent email pattern — buyer lookups by email are common.
idx_showing_time	showing_appointment / scheduled_time	Day/week/month calendar views always filter showings by a scheduled_time range.
idx_offer_listing	offer / listing_id	"Show all offers for this listing" is queried on every listing detail page load.
idx_offer_status	offer / status	The contract creation workflow filters offers by accepted status.

6. Views

View 1: active_listings_with_neighborhood

What it represents: A denormalized snapshot of every currently active listing, enriched with property details, neighborhood scores, and agent contact information.

Why it is useful: This is the most frequently executed query in the application — every Listings page and Dashboard load triggers it. Packaging it as a view centralizes the four-table join logic, keeps API route code clean, and ensures consistency across all callers.

Typical use case: GET /api/reports/active-listings and the Listings search page both query this view.

SQL Definition:

```
CREATE OR REPLACE VIEW active_listings_with_neighborhood AS
SELECT
    l.listing_id, l.list_price, l.list_date, l.description,
    p.address, p.city, p.zip_code, p.square_feet,
    p.bedrooms, p.bathrooms, p.year_built, p.property_type,
    n.name AS neighborhood_name, n.walkability_score,
    n.school_rating, n.flood_zone,
    CONCAT(a.first_name, ' ', a.last_name) AS agent_name,
    a.phone AS agent_phone,
    a.email AS agent_email
FROM listing l
JOIN property     p ON l.property_id     = p.property_id
JOIN neighborhood n ON p.neighborhood_id = n.neighborhood_id
JOIN agent        a ON l.agent_id        = a.agent_id
WHERE l.status = 'active';
```

View 2: agent_days_on_market

What it represents: An aggregated performance summary per agent showing how quickly their listings close, calculated as the difference between list_date and closing_date for all closed contracts.

Why it is useful: Days-on-market is the primary KPI for agent performance at a brokerage. Computing this on every report request requires an expensive multi-table join with aggregation — defining it as a view allows the Reports page to call it with a simple SELECT *.

Typical use case: GET /api/reports/agent-performance renders a sorted performance table on the Reports page.

SQL Definition:

```
CREATE OR REPLACE VIEW agent_days_on_market AS
SELECT
    a.agent_id,
```

```
    CONCAT(a.first_name, ' ', a.last_name) AS agent_name,
    a.brokerage,
    COUNT(l.listing_id)                                  AS total_closed,
    ROUND(AVG(DATEDIFF(c.closing_date, l.list_date)), 1) AS
avg_days_on_market,
    MIN(DATEDIFF(c.closing_date, l.list_date))           AS min_days,
    MAX(DATEDIFF(c.closing_date, l.list_date))           AS max_days
FROM agent a
JOIN listing  l ON a.agent_id   = l.agent_id
JOIN contract c ON l.listing_id = c.listing_id
WHERE c.status = 'closed'
GROUP BY a.agent_id, a.first_name, a.last_name, a.brokerage;
```

7. Transaction Scenario

Business Scenario

When a buyer's offer is accepted on a listing, two database changes must occur together:

- A new row is inserted into offer with status = 'accepted'.
- The corresponding listing row is updated from status = 'active' to status = 'under_contract'.

Why Atomicity Is Required

These two operations are causally linked. If the offer insert succeeds but the listing update fails (e.g., a network error or constraint violation), the database would contain an accepted offer with no corresponding status change on the listing — the listing would appear still active and could receive further offers. Conversely, if the listing status changes but the offer row is never written, there is no record of why the listing is under contract.

What Could Go Wrong Without a Transaction

- Race condition: two buyers could have their offers accepted on the same listing simultaneously, both reading status = 'active' before either UPDATE commits.
- Partial failure: an orphaned under_contract listing with no matching accepted offer would break the contract creation workflow downstream.

SQL Implementation (Stored Procedure)

```
CREATE PROCEDURE accept_offer(
    IN p_listing_id      INT,
    IN p_buyer_id        INT,
    IN p_offer_price     DECIMAL(12,2),
    IN p_contingencies   TEXT,
    IN p_expiration_date DATE
)
BEGIN
    DECLARE rows_updated INT DEFAULT 0;
    START TRANSACTION;

    INSERT INTO offer (listing_id, buyer_id, offer_price, contingencies,
                       expiration_date, status, submitted_at)
    VALUES (p_listing_id, p_buyer_id, p_offer_price, p_contingencies,
            p_expiration_date, 'accepted', NOW());

    UPDATE listing
    SET status = 'under_contract'
    WHERE listing_id = p_listing_id AND status = 'active';

    SET rows_updated = ROW_COUNT();
```

```
    IF rows_updated = 0 THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Listing is not active — rolled back.';
    ELSE
        COMMIT;
    END IF;
END;
```

ROLLBACK condition: If UPDATE listing affects 0 rows (the listing is already under_contract, sold, or withdrawn), the entire transaction is rolled back — the offer insert is undone and an error is returned to the caller. No partial state is ever persisted.

8. Application Overview

Type

Full-stack web application — React SPA (Single Page Application) with a Node.js/Express REST API backend.

Architecture

```
Browser  (React 18 + Vite)
    │  HTTP / REST  (JSON)
    ▼
Node.js / Express  (port 5000)
    │  mysql2/promise connection pool
    ▼
MySQL 8  (InnoDB)  —  class server  167.71.90.83
```

Technologies

- Frontend: React 18, React Router v6, Vite
- Backend: Node.js v24, Express, mysql2/promise, dotenv, cors
- Database: MySQL 8 (InnoDB), hosted on class server at 167.71.90.83
- DB credentials managed via .env file (not committed to source control)

Pages & CRUD Coverage

Page / Route	Tables Accessed	Operations
Dashboard /	listing, property, agent, neighborhood	Read
Listings /listings	listing, property, agent, neighborhood	Read + Search/Filter
Listing Detail /listings/:id	listing, property, agent, neighborhood, offer, buyer	Read; Create offer
Properties /properties	property, neighborhood	Create, Read, Update, Delete
Agents /agents	agent	Create, Read, Update, Delete
Buyers /buyers	buyer	Create, Read, Update, Delete
Showings /showings	showing_appointment, listing, buyer, agent	Create, Read, Update, Delete
Contracts /contracts	contract, listing, offer	Create, Read, Update, Delete
Inspections /inspections	inspection, listing	Create, Read, Update, Delete
Reports /reports	active_listings_with_neighborhood (View 1), agent_days_on_market (View 2)	Read (reporting)

Additional Features

- Search & filter on Listings page: status, agent, price range, neighborhood, minimum bedrooms
- Form validation: all POST/PUT routes validate required fields server-side and return HTTP 400 with a descriptive error before any DB query executes (prevents bad data from reaching the database)
- Parameterized queries throughout: all user input is bound via ? placeholders in mysql2, preventing SQL injection
- Transaction unified in app and database: POST /api/offers/accept calls the accept_offer stored procedure directly, so ROLLBACK behavior is handled once at the database level with no duplicated transaction logic in the application.
- Reporting view: Reports page queries both database views and renders them as summary tables

9. Known Limitations and Future Work

Current Limitations

- No authentication: the application has no login system. Any user can create, update, or delete any record. Production use would require JWT-based auth with role separation (agent vs. buyer vs. admin).
- No pagination: large result sets are returned in full. At production scale this would cause slow page loads and excess memory use.
- No image storage: properties and listings have no photo upload capability. inspection.report_url stores a URL string with no upload mechanism.

Design Trade-offs

- preferences (buyer) and contingencies (offer) are TEXT columns rather than normalized sub-tables — a deliberate choice for flexibility and scope.
- report_url on inspection stores a URL string, deferring file storage to an external service (e.g., S3).
- contract.listing_id is stored directly alongside offer_id as a documented efficiency-oriented implementation choice for simpler contract queries, while consistency is still preserved through foreign key constraints.

Future Extensions

- User authentication and role-based access control
- Pagination and cursor-based API responses
- Photo gallery uploads per listing (S3 + URL reference in DB)
- Email/SMS notifications when a showing is scheduled or offer status changes
- Advanced analytics dashboard — price trend charts per neighborhood, monthly volume reports
- Mobile-responsive UI improvements